

Book Service Documentation

Overview

The Book Service is a comprehensive Angular service that manages all book-related operations in the BookVerse frontend application. It serves as the primary interface between the frontend components and the backend API, handling book CRUD operations, search functionality, category management, sales analytics, and data transformation.

Service Configuration

- ****Injectable****: Root-level service available throughout the application
- ****Base API URL****: `http://localhost:8090/api` (API Gateway)
- ****Primary Endpoints****:
 - Books: `/api/books`
 - Books by Category: `/api/books/category`

Core Functionality

1. Basic Book Operations

Get All Books

```typescript

getAllBooks(): Observable<BookModel[]>

```

- Retrieves all active books from the backend
- Maps backend data to frontend BookModel format
- Used in: Book listings, admin panels

Get Book by ID

```typescript

getBookById(id: string | number): Observable<BookModel>

getBookWithRelations(id: string | number): Observable<BookModel>

```

- Fetches individual book details
- `getBookWithRelations` includes categories and images
- Used in: Book detail pages, edit forms

Create Book

```typescript

createBook(book: Omit<BookModel, 'id'>): Observable<BookModel>

createBookWithRelations(bookData: any): Observable<BookModel>

```

- Creates new books in the system
- `createBookWithRelations` handles categories and images
- Used in: Admin book creation forms

Update Book

```typescript

updateBook(id: string | number, book: Partial<BookModel>):

Observable<BookModel>

patchBook(id: string | number, bookUpdates: Partial<BookModel>):

Observable<BookModel>

```

- Updates existing books (full or partial updates)
- Maps frontend data to backend format
- Used in: Admin book editing

Delete Book

```
```typescript
deleteBook(id: string | number): Observable<any>
restoreBook(id: string | number): Observable<any>
```
```

- Soft delete functionality
- Restore capability for deleted books
- Used in: Admin book management

2. Search and Discovery

Search Books

```
```typescript
searchBooks(query: string, filters?: any): Observable<{ results:
BookModel[], total: number }>
```
```

- ****Features****:
 - Text-based search by title, author, description
 - Category filtering
 - Price range filtering (frontend)
 - Returns paginated results with total count
- ****Filters Supported****:
 - ``categories``: Array of category names
 - ``maxPrice``: Maximum price filter
- Used in: Search pages, product discovery

Get Similar Books

```
```typescript
getSimilarBooks(bookId: string | number): Observable<BookModel[]>
```
```

- Finds books similar to a given book
- Based on categories, author, or other criteria
- Used in: Book detail pages, recommendations

3. Category and Section Management

Home Page Sections

```
```typescript
getNewlyLaunchedBooks(): Observable<BookWithSales[]>
getBestSellers(): Observable<BookWithSales[]>
getSpecialOffers(): Observable<BookWithSales[]>
```
```

- Retrieves books for homepage sections
- Includes sales data for each book
- Maps to backend sales categories:
 - ``NEWLY_LAUNCHED``
 - ``BEST_SELLING``
 - ``SPECIAL_OFFERS``

Section Data Management

```
```typescript
getSectionData(section): Observable<{id: string}[]>
updateSectionData(section, bookIds: string[]): Observable<{id: string}[]>
addBookToSection(section, bookId: string): Observable<{id: string}[]>
removeBookFromSection(section, bookId: string): Observable<{id:
string}[]>
```
```

- Manages which books appear in homepage sections
- Supports dynamic section updates

- Used in: Admin section management

Category-Based Retrieval

```typescript

getBooksByCategory(): Observable<BookCategoryData[]>

getBooksByCategoryType(category): Observable<BookWithSales[]>

addBookToCategory(bookId: string, category): Observable<BookCategoryData>

removeBookFromCategory(bookId: string, category): Observable<any>

```

- Manages books in specific categories

- Tracks sales data per category

- Categories: 'newly launched', 'highly rated', 'special offers'

4. Sales and Analytics

Sales Analytics

```typescript

getHighlySoldBooks(limit: number = 10): Observable<BookWithSales[]>

getLeastSoldBooks(limit: number = 10): Observable<BookWithSales[]>

```

- Retrieves books sorted by sales performance

- Excludes books with zero sales from least sold

- Used in: Admin dashboards, analytics

Stock Management

```typescript

updateBookStock(bookId: string | number, quantity: number):

Observable<any>

updateBookSalesCount(bookId: string, category):

Observable<BookCategoryData | null>

```

- Updates book inventory after purchases

- Tracks sales counts per category

- Used in: Order processing, inventory management

5. Admin-Specific Operations

Admin Book Management

```typescript

getAllBooksForAdmin(): Observable<BookModel[]>

getBooksByCategoryForAdmin(category): Observable<BookWithSales[]>

getAllBooksWithCategories(): Observable<{ book: BookModel; categories:

string[]; salesCount: number }[]>

```

- Includes inactive/deleted books

- Provides comprehensive book data with categories and sales

- Used in: Admin dashboards, book management

Separate Updates for Complex Relations

```typescript

updateBookCategories(id: string | number, categoryIds: number[]):

Observable<BookModel>

updateBookImages(id: string | number, images: any[]):

Observable<BookModel>

```

- Handles category and image updates separately

- Avoids JPA relationship complexities

- Used in: Admin book editing (advanced features)

Data Transformation

Backend to Frontend Mapping

The service includes comprehensive data mapping between backend and frontend formats:

`mapBackendBookToFrontend(backendBook: any): BookModel`

- Converts backend book data to frontend BookModel
- Handles image URL extraction
- Maps sales categories
- Provides backward compatibility properties
- Processes nested relationships (categories, images)

`mapFrontendBookToBackend(frontendBook: Partial<BookModel>): any`

- Converts frontend data for backend consumption
- Formats dates properly (ISO strings)
- Handles stock field mapping
- Excludes complex relationships (handled separately)

Date Formatting

```typescript

formatDateForBackend(date: any): string | null

```

- Converts various date formats to ISO strings
- Handles array format: `[year, month, day, hour, minute]`
- Supports Date objects and string formats

Error Handling

Centralized Error Management

```typescript

private handleError<T>(operation = 'operation', result?: T)

```

- Re-throws errors to allow component-level handling
- Provides operation context for debugging
- Maintains error propagation chain

Key Features

1. Flexible ID Handling

- Supports both string and number IDs
- Automatic conversion for API consistency
- Utility method: `toStringId(id: string | number): string`

2. Sales Category Mapping

- Frontend: 'newlyLaunchedBooks', 'bestSellers', 'specialOffers'
- Backend: 'NEWLY_LAUNCHED', 'BEST_SELLING', 'SPECIAL_OFFERS'
- Bidirectional mapping support

3. Backward Compatibility

- Maintains legacy property names
- Supports both new and old data structures
- Gradual migration support

4. Comprehensive Search

- Multi-criteria search support
- Client-side and server-side filtering

- Fallback to getAllBooks when no criteria

Usage Examples

Basic Book Retrieval

```typescript

// Get all books

```
this.bookService.getAllBooks().subscribe(books => {
 this.books = books;
});
```

// Get specific book

```
this.bookService.getBookById('123').subscribe(book => {
 this.selectedBook = book;
});
```
```

Search Implementation

```typescript

// Search with filters

```
const filters = {
 categories: ['Fiction', 'Mystery'],
 maxPrice: 500
};
```

```
this.bookService.searchBooks('harry potter', filters).subscribe(result =>
{
 this.searchResults = result.results;
 this.totalResults = result.total;
});
```
```

Homepage Sections

```typescript

// Load homepage sections

```
this.bookService.getNewlyLaunchedBooks().subscribe(books => {
 this.newlyLaunched = books;
});
```

```
this.bookService.getBestSellers().subscribe(books => {
 this.bestSellers = books;
});
```
```

Admin Operations

```typescript

// Get books with full admin data

```
this.bookService.getAllBooksWithCategories().subscribe(data => {
 this.adminBooks = data;
});
```

// Update book

```
const updates = { price: 299, stockActual: 50 };
this.bookService.patchBook(bookId, updates).subscribe(updatedBook => {
 // Handle success
});
```
```

Dependencies

Required Imports

- `@angular/core` - Injectable decorator
- `@angular/common/http` - HTTP client and parameters
- `rxjs` - Observable operators and utilities
- `../models/book.model` - Type definitions

RxJS Operators Used

- `map` - Data transformation
- `catchError` - Error handling
- `switchMap` - Sequential operations
- `forkJoin` - Parallel operations
- `of` - Observable creation

Integration Points

Components Using Book Service

- **Home Component**: Homepage sections
- **Book Detail Component**: Individual book data
- **Search Component**: Book search and filtering
- **Admin Components**: Book management
- **Cart/Order Components**: Stock updates

Related Services

- **Review Service**: Customer ratings and reviews
- **Category Service**: Book categorization
- **Order Service**: Purchase processing
- **Image Service**: Book image management

Performance Considerations

Optimization Strategies

1. **Data Mapping**: Efficient transformation between formats
2. **Error Propagation**: Allows component-level error handling
3. **Batch Operations**: Multiple updates in single calls
4. **Caching**: Relies on HTTP interceptors for caching
5. **Lazy Loading**: Supports pagination and filtering

Memory Management

- Uses Observables for automatic cleanup
- Minimal data retention in service
- Efficient data transformation pipelines

Future Enhancements

Planned Improvements

1. **Enhanced Search**: Full-text search, faceted filtering
2. **Real-time Updates**: WebSocket integration for live data
3. **Caching Layer**: Service-level caching for frequently accessed data
4. **Batch Operations**: Bulk book operations for admin
5. **Advanced Analytics**: More detailed sales and performance metrics

Technical Debt

1. **Legacy Properties**: Gradual removal of backward compatibility
2. **Separate Endpoints**: Consolidation of category and image updates
3. **Error Handling**: More granular error types and handling
4. **Type Safety**: Stricter typing for backend responses

Conclusion

The Book Service provides a comprehensive, well-structured interface for all book-related operations in the BookVerse application. It successfully abstracts backend complexity while providing flexible, type-safe methods for frontend components. The service demonstrates good separation of concerns, proper error handling, and maintainable code structure suitable for a production e-commerce application.